

บทที่ 4

การพัฒนาชิ้นงานของโครงการ

เนื้อหาในบทนี้จะกล่าวถึงการพัฒนาชิ้นงาน ซึ่งประกอบด้วย ส่วนขยายของโปรแกรม Gaim (IsagQ) และส่วนของเซิร์ฟเวอร์ (IsagMQ) ซึ่งนำ OpenSSL Library มาช่วยในการเขียนโปรแกรม โดยกล่าวถึงการเตรียมสภาพแวดล้อมก่อนการพัฒนา และกระบวนการพัฒนาจนสำเร็จ

4.1 การนำโอเพนเอสแอลไลบรารี (OpenSSL Library) มาใช้ในโครงการ

โอเพนเอสแอลไลบรารีเป็น opensource ไลบรารีมีความสามารถในการทำงานในด้านการเข้ารหัสลับและถอดรหัสการสร้างใบรับรองสิทธิ์รวมไปถึงPKI,การสร้างแอปพลิเคชันที่มีการใช้โปรโตคอลSSLหรือTLS และมีความสามารถที่จะใช้แบบ crsoss-platform สามารถใช้ได้ทั้งวินโดวส์และยูนิกซ์ โอเพนเอสแอลไลบรารีนี้ใช้ได้ทั้ง c และ c++ แต่ก็สามารถใช้จาก command line ได้ด้วย รวมถึง Python ,Perl ,PHP

ในการสร้าง IsagQ และ IsagMQ นั้นมีการใช้ไลบรารีหลักๆ คือ การเข้ารหัสด้วย RSA การสร้างใบรับรองขอใบรับรองสิทธิ์,ใบรับรองสิทธิ์,การจัดการใบรับรองสิทธิ์และการเชื่อมต่อระหว่าง IsagQ และ IsagMQ ด้วย SSL

อาร์เอสเอไลบรารี(RSA Library)

ฟังก์ชันการทำงานรวมถึงโครงสร้างของ RSA จะถูกใช้โดย include ไฟล์ openssl/rsa.h ซึ่งในโครงสร้างของ RSA นั้นจะประกอบด้วยค่าต่างๆดังนี้

```
typedef struct    rsa_st
{
    BIGNUM *p;
    BIGNUM *q;
    BIGNUM *n;
    BIGNUM *e;
    BIGNUM *d;
}RSA;
```

ค่า p และ q เป็นเลขจำนวนเฉพาะขนาดใหญ่ทั้งคู่แล้วนำมาคูณกันได้ค่า n ซึ่งจะเรียกว่า public modulus ค่า e (public exponent) จะเลือกอยู่ในค่าแบบสุ่มในช่วง (p-1)(q-1) ซึ่งเป็น relatively prime การสร้างกุญแจของอาร์เอสเอ (RSA Key) สร้างโดยใช้ฟังก์ชันต่อไปนี้

```
RSA *RSA_generate_key ( int num unsigned long e,void(*callback)(int,int,void *),void *cb_arg)
```

- num คือ ค่าความแข็งแรงของกุญแจ อย่างต่ำคือ 1024 ควรจะใช้ 2048
- e คือ ค่า public exponent แต่แนะนำให้ใช้ RSA_3,RSA_F4
- callback คือ ฟังก์ชันที่จะเรียกเพื่อคู่สถานะขณะสร้าง p,q
- cb_arg คือค่าอาร์กิวเมนต์ของ callback

การเข้ารหัสด้วยอาร์เอสเอ (RSA Encryption)

Int RSA_public_encrypt(int flen,unsigned char *from,unsigned char *to,RSA *rsa,int padding)

- flen คือ ค่าความยาวเป็น ไบท์ของข้อมูลในบัฟเฟอร์ที่จะถูกเข้ารหัส
- from คือ บัฟเฟอร์ข้อมูลที่จะนำไปเข้ารหัสลับ
- to คือ บัฟเฟอร์ที่จะรับข้อมูลที่เข้ารหัสแล้วโดยจะต้องมีขนาดที่ใหญ่พอ ซึ่งสามารถคำนวณโดยเรียกใช้ RSA_size และส่งออบเจกต์ RSA เป็นอาร์กิวเมนต์
- rsa คือ ออบเจกต์ของRSA
- padding ชนิดของแพดดิ้งซึ่งมี4ชนิด คือ

RSA_PKCS1_PADDING,RSA_PKCS1_OAEP_PADDING,RSA_SSLV23_PADDING,RSA_NO_PADDING

การถอดรหัสด้วยอาร์เอสเอ (RSA Decryption)

Int RSA_private_decrypt(int flen,unsigned char *from,unsigned char *to,RSA *rsa,int padding)

- flen คือ ค่าความยาวเป็น ไบท์ของข้อมูลในบัฟเฟอร์ที่จะถูกเข้ารหัส
- from คือ บัฟเฟอร์ข้อมูลที่จะนำไปถอดรหัสลับ
- to คือ บัฟเฟอร์ที่จะรับข้อมูลที่ถอดรหัสแล้วโดยจะต้องมีขนาดที่ใหญ่พอ ซึ่งสามารถคำนวณโดยเรียกใช้ RSA_size และส่งออบเจกต์ RSA เป็นอาร์กิวเมนต์
- rsa คือ ออบเจกต์ของRSA
- padding ชนิดของแพดดิ้งซึ่งมี4ชนิด คือ

RSA_PKCS1_PADDING,RSA_PKCS1_OAEP_PADDING,RSA_SSLV23_PADDING,RSA_NO_PADDING

การสร้าง SSL แอปพลิเคชัน โดย Openssl

การสร้าง SSL แอปพลิเคชัน โดย Openssl มีขั้นตอนในการสร้างหลักๆ 3 ขั้นตอนคือ

1. การเลือกเอสเอสแอลเวอร์ชัน(SSL version selection)
2. การระบุตัวตนเพียร์ (Peer Authentication)
 1. ตัวเลือกของเอสเอสแอลและชุดของไซเฟอร์(SSL Options and Cipher Suites)

ขั้นตอนที่ 1 การเลือกเอสเอสแอลเวอร์ชัน(SSL version selection)

ในขั้นตอนนี้จะเกี่ยวกับออบเจกต์หลักๆอยู่3ชนิดคือ SSL_METHOD,SSL_CTX,และSSL ซึ่ง SSL_METHOD คือ ตัวแทนของการสร้างเอสเอสแอลฟังก์ชัน ซึ่งมีดังนี้

SSLv2_method	Returns a pointer to SSL_METHOD for generic SSL Version 2
SSLv2_client_method	Returns a pointer to SSL_METHOD for an SSL Version 2 client
SSLv2_server_method	Returns a pointer to SSL_METHOD for an SSL Version 2 server
SSLv3_method	Returns a pointer to SSL_METHOD for generic SSL Version 3
SSLv3_client_method	Returns a pointer to SSL_METHOD for an SSL Version 3 client
SSLv3_server_method	Returns a pointer to SSL_METHOD for an SSL Version 3 server
TLSv1_method	Returns a pointer to SSL_METHOD for generic TLS Version 1
TLSv1_client_method	Returns a pointer to SSL_METHOD for a TLS Version 1 client
TLSv1_server_method	Returns a pointer to SSL_METHOD for a TLS Version 1 server
SSLv23_method	Returns a pointer to SSL_METHOD for generic SSL/TLS
SSLv23_client_method	Returns a pointer to SSL_METHOD for an SSL/TLS client
SSLv23_server_method	Returns a pointer to SSL_METHOD for an SSL/TLS server

ตารางที่ 4.1 เวอร์ชันเอสเอสแอล (SSL)

SSL_CTX คือออบเจกต์สำหรับการเชื่อมต่อแบบเอสเอสแอล ซึ่งคอนเท็กซ์(context) จะอนุญาตให้เราปรับแต่งค่าการเชื่อมต่อก่อนที่จะสร้างการเชื่อมต่อ เช่น เวอร์ชันโปรโตคอล, รายละเอียดของใบรับรองสิทธิ์ ซึ่งการสร้างออบเจกต์ชนิดนี้ใช้ SSL_CTX_new

ขั้นตอนที่ 2 การระบุตัวตนเพียร์ (Peer Authentication)

การพิสูจน์ตัวตนของใบรับรองสิทธิ์ผู้ที่ตรวจสอบจำเป็นต้องมีรายการของ CA ดังนั้นเราจึงต้องบอกแอปพลิเคชันว่ารายการนั้นอยู่ที่ไหนเพื่อที่จะพิสูจน์ตัวตนของเพียร์ การที่จะใช้ที่เก็บรายการของ CA จำเป็นที่จะต้องเพิ่มส่วนการเตรียมออบเจกต์ SSL_CTX ด้วย

```
int SSL_CTX_load_verify_locations(SSL_CTX *ctx, const char *CAfile, const char *CApath).
```

ctx คือ เอสเอสแอลคอนเท็กซ์ออบเจกต์ที่จะใช้ใบรับรองสิทธิ์

CAfile คือ ชื่อไฟล์ใบรับรองสิทธิ์ในรูปแบบของ PEM

CApath คือ ชื่อไดเรกทอรีที่เก็บไฟล์ใบรับรองสิทธิ์

ขั้นตอนที่ 3 ตัวเลือกของเอสเอสแอลและชุดของไซเฟอร์ (SSL Options and Cipher Suites)

ฟังก์ชัน SSL_CTX_set_options ทำให้ผู้ใช้สามารถใช้งาน bug workaround ซึ่งสามารถทำให้เอสแอลแอปพลิเคชันสามารถเชื่อมต่อกับเพียร์ที่มีปัญหาได้หรือสามารถจำกัดระดับความปลอดภัยของการเชื่อมต่อเอสเอสแอล เช่น เลือกใช้ SSL_OP_NO_SSLv2 ซึ่งการใช้ทางเลือกนี้จะทำให้การเชื่อมต่อนี้จะไม่รับโปรโตคอลเอสเอสแอลเวอร์ชัน 2

การเลือกชุดของไซเฟอร์

ชุดของไซเฟอร์คือชุดของอัลกอริทึมที่เอสเอสแอลใช้ในการเชื่อมต่อแบบปลอดภัย ซึ่งจะใช้ฟังก์ชัน SSL_CTX_set_cipher_list ทำให้เราสามารถใส่รายชื่อของชุดของไซเฟอร์ได้ โดยรายชื่อของชุดของไซเฟอร์จะอยู่ในรูปแบบพิเศษคือ คั่นด้วย : ตัวอย่างเช่น

“ALL:!ADH:!LOW:!EXP:!MD5:@strength”

- ALL คือ การใช้ตัวแทนของทุกเอาต์กอร์ธึมมาคอมบิเนชันกัน
- ! คือ การยกเว้นอัลกอร์ธึม
- ADH คือ การใช้อัลกอร์ธึม Diffie Helman
- Low คือ การอ้างถึงอัลกอร์ธึมที่ใช้คีย์ 64 บิตหรือ 56 บิต
- EXP คือ การอ้างถึงอัลกอร์ธึมที่ใช้คีย์ 56 บิตหรือ 40 บิตแบบคลิปปี้ด
- @strength คือ การเรียงไซเฟอร์ตามขนาดคีย์

4.2 การพัฒนาส่วนขยายบน Gaim (IsagQ)

ในการพัฒนาปลั๊กอินของGAIMแบ่งได้เป็น2ภาษาคือภาษาซีและภาษาเพิร์ลซึ่งในโครงการจะใช้ภาษาซีในการพัฒนาส่วนขยายต้องใช้ซอร์สโค้ดของ Gaim ในการคอมไพล์โดยจะมี Gaim apiรองรับการทำงานต่างๆ และสามารถนำไลบรารีจากที่อื่นมาใช้ได้

4.2.1 การเตรียมสภาพแวดล้อมในการพัฒนาส่วนขยายของ Gaim

การเตรียมสภาพแวดล้อมในการพัฒนาส่วนขยายของ Gaim มีขั้นตอนดังนี้

- 1) ดาวน์โหลดซอร์สโค้ดของ Gaim เวอร์ชัน 1.5
- 2) เข้าไปที่โฟลเดอร์ Gaim-1.5.0 แล้วสั่ง ./configure
- 3) ในขั้นนี้เราจะได้ Make file ขึ้นมา
- 4) ถ้าต้องการincludeไลบรารีใดๆที่ต้องใช้ในส่วนขยายให้เข้าไปที่Gaim-1.5.0/plugins/Makefile เข้าไปให้ระบุในไฟล์นี้ เช่น เราต้องการใช้ไลบรารีของ Openssl ซึ่งจะเก็บไว้ที่ /usr/local/include ก็เพิ่ม -I/usr/local/include
- 5) เมื่อต้องการคอมไพล์
 - 5.1 Window สั่ง make -f makefile.mingw name.dll
 - 5.2 Unix/linux สั่ง make name.so
 ซึ่ง **name** คือชื่อไฟล์ .c ที่ต้องการคอมไพล์
- 6) เมื่อเสร็จสิ้นคำสั่ง make ให้คัดลอกไฟล์ไปยังโฟลเดอร์ที่เก็บไลบรารีของ Gaim ไว้เช่น /usr/X11R6/lib/gaim

4.2.2 การพัฒนาส่วนขยาย IsagQ

โครงสร้างไฟล์ส่วนขยายหลักของ Gaim

ในไฟล์หลักของส่วนขยายนั้นจะมีรูปแบบที่ตายตัวและจำเป็นที่จะต้องทำตามรูปแบบเพื่อที่จะสามารถใช้ส่วนขยายได้โดยแบ่งออกได้ดังนี้

1 ส่วนที่หนึ่งเป็นการ include file และ #define GAIM_PLUGINS ซึ่งต้องมีไฟล์อย่างน้อยดังนี้

```
#define GAIM_PLUGINS                #ต้องประกาศในทุกปลั๊กอิน

#include <glib.h>

#include "notify.h"

#include "plugin.h"

#include "version.h"

static gboolean

plugin_load(GaimPlugin *plugin) {    #ฟังก์ชันนี้จะทำงานตอนที่โหลดปลั๊กอิน

    return TRUE;

}
```

2 ส่วนที่สองคือการตั้งค่าต่างๆ ของส่วยขยาย

```
static GaimPluginInfo info = {    #ฟังก์ชันนี้จะเป็นรายละเอียดที่แสดงในconfig window

                                (จำเป็นต้องมี)
```

```
    GAIM_PLUGIN_MAGIC,           #เพื่อหลีกเลี่ยง crash เวลาโหลด
    GAIM_MAJOR_VERSION,         #บอกเวอร์ชัน
    GAIM_MINOR_VERSION,         #บอกเวอร์ชัน
    GAIM_PLUGIN_STANDARD,       #ชนิดของปลั๊กอิน
    NULL,                       #ui requirement
    0,                          #plugin flags
    NULL,                       #plugin dependencie
    GAIM_PRIORITY_DEFAULT,      #ไพออร์ติตี้ของปลั๊กอิน
    "core-hello_world",         #ไอดี
    "Hello World!",             #ชื่อ
    VERSION,                    #เวอร์ชันของปลั๊กอิน
    "Hello World Plugin",       #รายละเอียดโดยย่อของปลั๊กอิน
    "Hello World Plugin",       #รายละเอียดของปลั๊กอิน
    NULL,                      #ชื่อและอีเมลล์
    GAIM_WEBSITE,               #เว็บ
    plugin_load,                #พอยน์เตอร์ชี้ไปยังฟังก์ชันที่จะโหลด
                                ตอนเริ่มปลั๊กอิน
    NULL,                      #พอยน์เตอร์ชี้ไปยังฟังก์ชันที่จะโหลด
                                ตอนปิดปลั๊กอิน
```

```

NULL,                                #พอยน์เตอร์ชี้ไปยังฟังก์ชันที่จะโหลด
                                      ตอนทำลายปลั๊กอิน
&ui_info,                            #พอยน์เตอร์ชี้ไปยัง UI struct

NULL,                                #พอยน์เตอร์ชี้ไปยัง GaimPluginLoaderInfo
                                      และGaimPluginProtocolInfo
NULL,                                #พอยน์เตอร์ชี้ไปยัง GaimPluginUiInfo
NULL                                 #พอยน์เตอร์ชี้ไปยังฟังก์ชันที่จะโหลดเวลาเรียกเมนู
};

3 . ส่วนที่ 3 จำเป็นต้องประกาศไว้เสมอ ถ้าไม่ประกาศจะไม่สามารถเรียกใช้ส่วนขยายได้

static void
init_plugin(GaimPlugin *plugin) {      #ฟังก์ชันที่เรียกเวลาตรวจสอบปลั๊กอิน
}

GAIM_INIT_PLUGIN(hello_world, init_plugin, info);  #เป็นมาโครที่ทุกปลั๊กอินต้องมี

```

การเขียนหน้าจอการตั้งค่าส่วนขยาย

เมื่อปลั๊กอินโหลดขึ้นมาเราสามารถที่จะสร้างหน้าจอการตั้งค่าได้โดยจะระบุไว้ในฟังก์ชัน GaimPluginInfo info ในอาร์กิวเมนต์ตัวที่ 19 โดยระบุเป็น pointer เช่น &ui_info ซึ่งเรียกฟังก์ชัน ui_info มีลักษณะดังนี้

```

static GaimGtkPluginUiInfo ui_info =
{
    get_config_frame
};

```

ซึ่งจะไปเรียกฟังก์ชัน get_config_frame ซึ่ง ui_info จะเป็นเพียงฟังก์ชันที่บอก Gaim ว่ามีการสร้างหน้าจอการตั้งค่าส่วนขยายซึ่งฟังก์ชันที่สร้างหน้าจอการตั้งค่าจริง ๆ คือ get_config_frame การตั้งค่าการทำงานเริ่มต้นของปลั๊กอิน

ก่อนที่ตัวปลั๊กอินจะเริ่มทำงานเราสามารถสั่งให้ตัวปลั๊กอินทำงานบางอย่างก่อนได้ โดยใช้ฟังก์ชัน plugin_load (GaimPlugin *handle) ซึ่ง IsagQ จะมีการทำงาน 2 อย่างในช่วงนี้คือ Init_pref() = สร้าง preference ที่สามารถบันทึกค่าลักษณะต่างๆของปลั๊กอินเช่น

- gaim_prefs_add_none สร้าง preference
- gaim_prefs_add_int บันทึกค่าลงไปใน preference ที่สร้างแล้ว
- gaim_prefs_get_int รับค่าจาก preference ที่บันทึกไว้

Init_cert() = สร้างกุญแจสาธารณะ, กุญแจส่วนตัว, ใบรับรองขอใบรับรองสิทธิ์, ใบรับรองสิทธิ์
สัญญาณบน Gaim (Gaim signal)

เมื่อผู้ใช้หรือคู่สนทนามีการตอบโต้กับโปรแกรม(event) ตัว Gaim ก็จะส่งสัญญาณ (Signal) ออกมาโดยเราสามารถจับสัญญาณเหล่านี้ได้โดยใช้ gaim_signal_connect แต่ก่อนที่จะจับสัญญาณได้จะต้องสร้าง handle สำหรับเชื่อมต่อก่อน ในตัว IsagQ จะมีการจับสัญญาณการทำงาน 2 อย่างคือ สัญญาณการส่งข้อความ "sending-im-msg" และสัญญาณการรับข้อความ "receiving-im-msg" ซึ่งมีรูปแบบการทำงานดังนี้

```
gaim_signal_connect(conv_handle, "sending-im-msg", isagq_plugin_handle,  
GAIM_CALLBACK(send_msg_cb), NULL);
```

ฟังก์ชันนี้จะมีอาร์กิวเมนต์ 5 ตัวคือ

- conv_handle คือ แอนเดิลของการสนทนาเนื่องจากฟังก์ชันที่จะทำงานนี้เกี่ยวข้องกับการสนทนา ซึ่ง แอนเดิลนี้สามารถเรียกใช้ได้จาก gaim_conversations_get_handle
- "sending-im-msg" คือ ชื่อของสัญญาณที่เราต้องการจะจับ
- isagq_plugin_handle คือ แอนเดิลของปลั๊กอิน IsagQ ซึ่งจะถูกสร้างในฟังก์ชัน plugin_load
- GAIM_CALLBACK(send_msg_cb) คือ การเรียกใช้ฟังก์ชัน send_msg_cb เมื่อมีสัญญาณการส่งข้อความ
- NULL คือ อาร์กิวเมนต์ของฟังก์ชัน send_msg_cb

การจัดการกับการส่งข้อความและการรับข้อความ

เมื่อเราสามารถจับสัญญาณของการส่งข้อความและการรับข้อความได้แล้วก็จะสามารถจัดการกับข้อความเหล่านั้นก่อนที่จะแสดงผลได้ โดยฟังก์ชันที่จะจัดการกับข้อความจะต้องมีอาร์กิวเมนต์ต่อไปนี้

- GaimAccount *account อาร์กิวเมนต์นี้จะมีรายละเอียดของผู้ใช้งานที่ได้รับหรือส่งข้อความนั้นๆ เช่น ชื่อผู้ใช้, โปรโตคอลที่ใช้
- char *who อาร์กิวเมนต์นี้คือ ชื่อของคู่สนทนา
- char **message คือ ข้อความที่ได้รับหรือส่ง

เมื่อเราจัดการกับข้อความเสร็จแล้วก็จะสามารถนำส่งหรือแสดงผลได้ โดยการส่งข้อความจะใช้ serv_send_im(server-send-im) หรือถ้าต้องการส่งอย่างอื่นก็ได้เช่น serv_send_file serv_send_typing โดย serv_send_im มีอาร์กิวเมนต์ต่างๆดังนี้

- GaimConnection * คือ ตัวชี้ไปยังการเชื่อมต่อกับคู่สนทนาซึ่งจะเป็น attribute อยู่ในตัวแปรชนิด GaimAccount ชื่อ gc
- Const char* คือ ชื่อของคู่สนทนา

- Const char* คือ ข้อความที่ต้องการส่ง
- GaimMessageFlags คือ ชนิดข้อความ เช่น GAIM_MESSAGE_SEND, GAIM_MESSAGE_SYSTEM

เช่น (acct->gc, name, cipher_msg, GAIM_MESSAGE_AUTO_RESP)

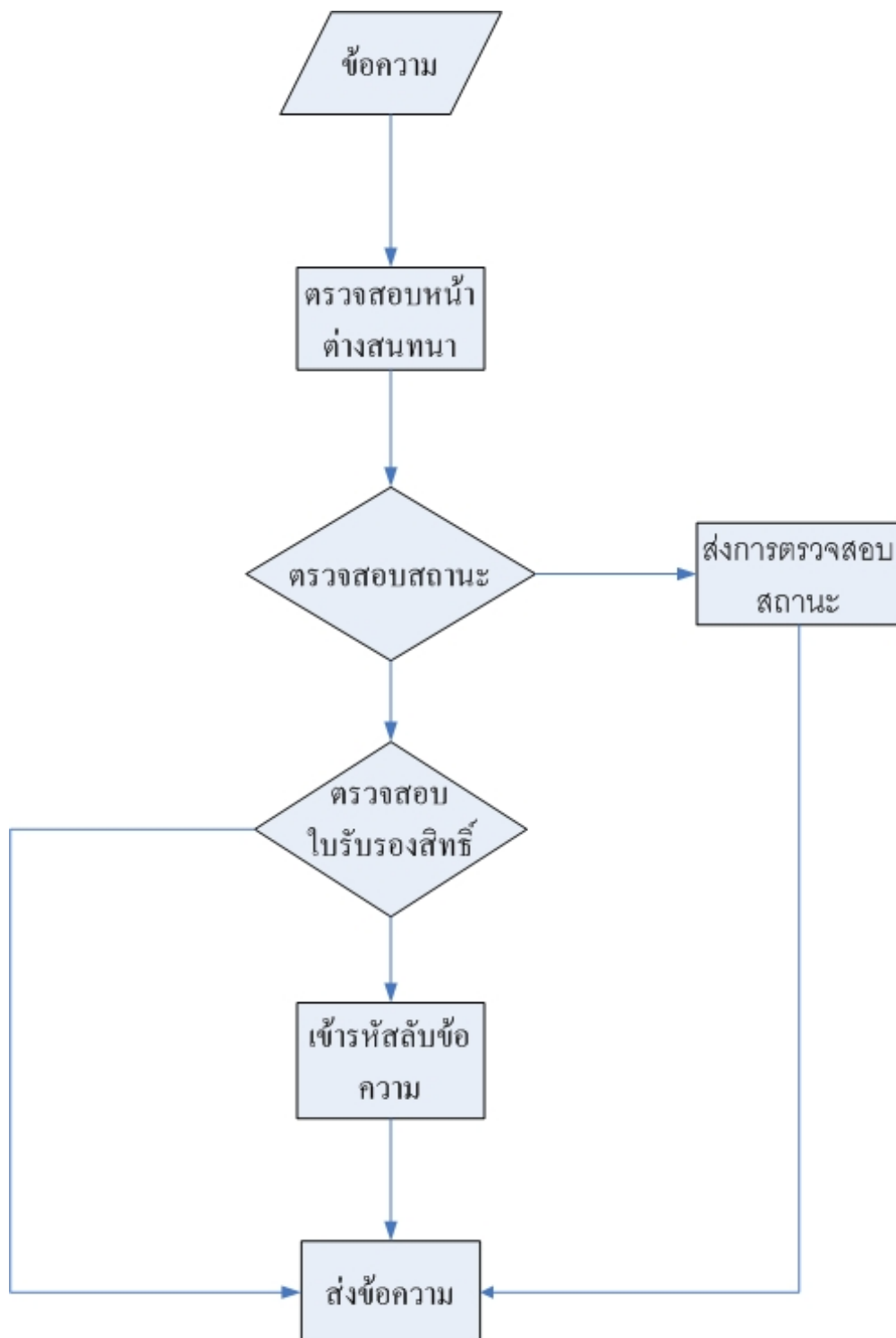
ทางการแสดงผลข้อความก็จะใช้ gaim_conv_im_write โดยมีอาร์กิวเมนต์ดังนี้

- GaimConvIm คือ ตัวชี้ไปยังหน้าต่างสนทนานั้นๆ
- Const char* คือ ชื่อของผู้สนทนา
- Const char* คือ ข้อความที่จะแสดงผล
- GaimMessageFlags คือ ชนิดของข้อความ
- time_t คือ การห้วงเวลาก่อนที่จะแสดงผล

เช่น gaim_conv_im_write(GAIM_CONV_IM(conv), *who, decrypt_msg, GAIM_MESSAGE_RECV, time((time_t) NULL))

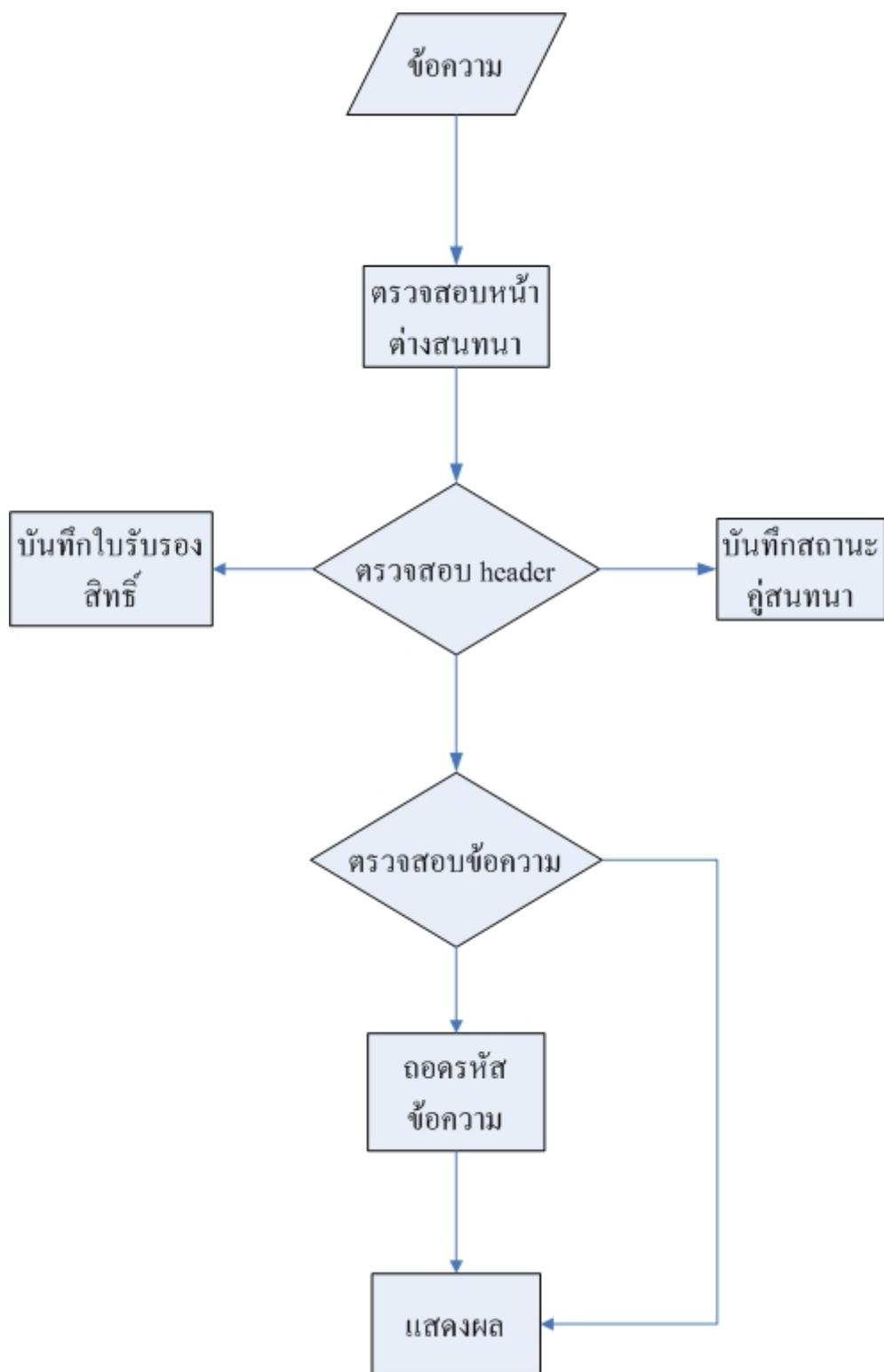
4.2.3 กระบวนการรับ-ส่งข้อความของ IsagQ

ขั้นตอนการส่งข้อความของ IsagQ



รูปที่ 4.1 ขั้นตอนการส่งข้อความของ IsagQ

ขั้นตอนการรับข้อความของ IsagQ



รูปที่ 4.2 ขั้นตอนการรับข้อความของ IsagQ

4.3 การพัฒนาส่วนเซิร์ฟเวอร์ (IsagMQ)

Certificate Authority เป็นบทบาทของส่วนเซิร์ฟเวอร์ ซึ่งจะสร้าง Root CA และ Server CA ซึ่ง Server CA จะเป็นผู้ sign ให้กับ Client หน้าที่ของ CA คือ การออกใบรับรองสิทธิ์ การเรียกคืนใบรับรองสิทธิ์ การเก็บข้อมูลของผู้ขอใบรับรองสิทธิ์ และการเก็บข้อมูลของใบรับรองสิทธิ์ที่ถูกเรียกคืนหรือหมดอายุแล้ว

4.3.1 การสร้างผู้ออกใบรับรองสิทธิ์ (Certification Authority)

เซิร์ฟเวอร์IsagMQพัฒนาบนระบบปฏิบัติการลินุกซ์ FreeBSD โดยต้องติดตั้งไลบรารี Openssl มีหน้าที่หลักคือ Certification Authority ซึ่งจัดการเรื่องใบรับรองสิทธิ์ การสร้าง CA มีดังนี้ โดยอย่างแรกเราสร้างโฟลเดอร์เพื่อเก็บข้อมูลทั้งหมดของCANั่นคือใบรับรองสิทธิ์และ CRL จากนั้นเราจะสร้างไฟล์ 3 ไฟล์ ไฟล์แรกคือไฟล์ serial ซึ่งเก็บ serial number สุดท้ายที่ถูกใช้ในการออกใบรับรองสิทธิ์เพื่อไม่ให้ serial numberซ้ำกัน ไฟล์ที่สองคือ ไฟล์ index.txt ซึ่งเก็บฐานข้อมูลของใบรับรองสิทธิ์ซึ่งออกโดยCA และไฟล์สุดท้ายคือไฟล์ configuration ซึ่งเก็บข้อมูลเกี่ยวกับวิธีการออกใบรับรองสิทธิ์

- ไฟล์ configuration มีดีฟอลท์อยู่แล้วเมื่อลง Openssl Library แต่เนื่องจากโครงงานนี้ต้องกำหนดค่าต่างๆเองจึงสร้างไฟล์ configuration ขึ้นมาใหม่ดังนี้

```
[ ca ]
default_ca          = exampleca

[ exampleca ]
dir                 = /root/careCA
certificate          = $dir/rootcert.pem
database            = $dir/index.txt
new_certs_dir       = /root/careClient
private_key          = $dir/rootkey.pem
serial              = $dir/serial
crl                 = $dir/crl.pem #the current CRL
crl_dir             = $dir/crl
default_crl_days    = 7
default_days        = 365
```

```
default_md          = md5
policy              = exampleca_policy
x509_extensions     = certificate_extensions
```

```
[ exampleca_policy ]
```

```
commonName          = supplied
stateOrProvinceName = supplied
countryName          = supplied
emailAddress         = supplied
organizationName     = supplied
organizationalUnitName = optional
```

```
[ certificate_extensions ]
```

```
basicConstraints     = CA:false
```

```
[ req ]
```

```
default_bits         = 2048
default_md           = md5
```

```
prompt = no
```

```
x509_extensions      = root_ca_extensions
```

```
[ root_ca_extensions ]
```

```
basicConstraints     = CA:true
```

- สร้าง ROOT CA โดย sign ด้วยตัวเอง

```
# openssl req -newkey rsa:1024 -sha1 -keyout rootkey.pem -out rootreq.pem
```

```
# openssl x509 -req -in rootreq.pem -sha1 -extfile careopenssl.cnf -signkey rootkey.pem -out  
rootcert.pem
```

```
#cat rootcert.pem rootkey.pem >root.pem
```

```
#openssl x509 -subject -issuer -noout -in root.pem
```

```
subject= /CN=care/emailAddress=pare_g7@hotmail.com issuer=  
/CN=care/emailAddress=pare_g7@hotmail.com
```

- สร้าง SERVER CA และ sign ด้วย ROOT CA

```
#openssl req -newkey rsa:1024 -sha1 -keyout serverCAkey.pem -out serverCAreq.pem
```

```
#openssl x509 -req -in serverCAreq.pem -sha1 -extfile careopenssl.cnf -CA root.pem -CAkey  
root.pem -CAcreateserial -out serverCAcert.pem
```

```
# cat serverCAcert.pem serverCAkey.pem rootcert.pem > serverCA.pem
```

```
# openssl x509 -subject -issuer -noout -in serverCA.pem
```

```
subject= /CN=pare_g7/emailAddress=pare_g72@hotmail.com
```

```
issuer= /CN=care/emailAddress=pare_g7@hotmail.com
```

- สร้าง SERVER'S CERTIFICATE และ sign ด้วยตัวมันเอง

```
# openssl req -newkey rsa:1024 -sha1 -keyout serverkey.pem -out serverreq.pem
```

```
# openssl x509 -req -in serverreq.pem -sha1 -extfile careopenssl.cnf -CA serverCA.pem -CAkey  
serverCA.pem -CAcreateserial -out servercert.pem
```

```
# cat servercert.pem serverkey.pem serverCAcert.pem rootcert.pem >server.pem
```

```
# openssl x509 -subject -issuer -noout -in server.pem
```

```
subject= /CN=pareserver/emailAddress=pare_g73@hotmail.com\x1B[D
```

```
issuer= /CN=pare_g7/emailAddress=pare_g72@hotmail.com
```

- การสร้าง Certificate Revocation List (CRL)

```
# openssl ca -gencrl -config /root/careCA/careopenssl.cnf -out /root/careCA/crl.crl
```

สร้าง CRL โดยกำหนด option ไว้ในไฟล์ careopenssl.cnf ดังนี้

Default crt days = 7
Default days = 365
Default md = md5

4.3.2 กระบวนการออกใบรับรองสิทธิ์

- CLIENT หรือ IsagQ สร้าง CERTIFICATE REQUEST

```
# gcc -o certReq certReq.c -lssl -lpthread  
# ./certReq priKey.pem certReq.pem
```

- CA สร้าง CLIENT CERTIFICATE

```
# gcc -o createCert createCert.c -lssl -lpthread  
# ./createCert certReq.pem clientCert.pem
```

- CA พิสูจน์ CLIENT CERTIFICATE

```
#gcc -o verifycert verifycert.c -lssl lpthread  
# ./verifycert.c clientCert.pem
```

Client Certificate จะถูก sign โดย Server CA ซึ่งจะเก็บรายละเอียดของไคลเอนต์
หมายเลขซีเรียล อนุญาตสาธารณะของไคลเอนต์ และ ลายเซ็นของ Server CA

```

isagmq# openssl x509 -in client.pem -noout -text
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      fb:54:de:11:9e:8e:35:26
    Signature Algorithm: sha1WithRSAEncryption
    Issuer: CN=care/emailAddress=pere_g7@hotmail.com
    Validity
      Not Before: Dec 23 12:15:44 2005 GMT
      Not After : Jan 22 12:15:44 2006 GMT
    Subject: CN=rach/emailAddress=rach@hotmail.com
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      RSA Public Key: (1024 bit)
        Modulus (1024 bit):
          00:b8:56:50:ca:1f:10:48:c2:5e:a3:2b:f0:30:06
:
          c3:2e:47:26:58:b5:4d:d9:30:2c:24:c8:86:84:3d
:
          cd:48:4b:2a:b0:96:f7:c7:55:ba:dc:cc:53:80:54
:
          ef:ba:61:74:14:bd:6f:84:c2:cd:f0:4b:46:ec:64
:
          30:17:66:56:2f:93:25:ec:d3:49:49:73:b3:94:0b
:
          c6:d2:f6:5e:f1:1a:58:4b:f7:eb:19:ed:0f:36:e1
:
          d2:28:b6:e8:b9:3b:65:16:16:ac:15:87:f0:58:79
:
          c8:4b:b4:ff:40:46:57:fb:00:c6:3e:9c:20:c6:96
:
          cb:be:31:38:3d:fe:1d:07:db
        Exponent: 65537 (0x10001)
    Signature Algorithm: sha1WithRSAEncryption
      64:a3:9a:5d:3d:fb:8f:6b:d5:86:53:88:14:70:0f:37:dc:37:
      58:d5:f6:ad:3f:86:21:a5:d6:7d:11:fe:88:bc:23:3d:7a:39:
      cf:ee:dc:93:3c:89:2e:5d:6b:8d:0d:10:e0:37:2f:98:17:d6:
      35:f9:5d:31:96:d7:2c:23:e9:0f:3c:b4:ec:7d:bb:51:4d:1e:
      a6:fc:02:7a:b8:9e:d9:00:89:fc:7c:6b:33:f7:52:5e:07:f2:
      8e:ac:0a:5e:f0:54:8b:ff:fe:e6:d4:21:cd:5e:fa:bf:d5:12:
      b1:62:f0:b9:ae:f3:1f:e5:ac:a9:4b:6a:e0:b2:1a:93:76:ee:
      23:fc

```

รูปที่ 4.3 client certificate

Certificate Revocation List (CRL) มีเพื่อเก็บ Serial Number ที่ถูกเรียกคืนโดย CA CRL มีอายุการใช้งาน มีการเก็บค่าเวอร์ชัน รายละเอียดของผู้สร้าง CRL วันที่ออกและหมดอายุของ CRL และ ลายเซ็นของผู้สร้างนั่นคือ Server CA

```

isagmq# openssl crl -in crl2.crl -noout -text
Certificate Revocation List (CRL):
    Version 1 (0x0)
    Signature Algorithm: md5WithRSAEncryption
    Issuer: /CN=care/emailAddress=pave_g7@hotmail.com
    Last Update: Dec 27 07:25:50 2005 GMT
    Next Update: Dec 27 08:25:50 2005 GMT
Revoked Certificates:
    Serial Number: FB54DE119E8E3523
        Revocation Date: Dec 25 17:05:41 2005 GMT
    Serial Number: FB54DE119E8E3524
        Revocation Date: Dec 25 17:03:04 2005 GMT
    Serial Number: FB54DE119E8E3525
        Revocation Date: Dec 25 16:29:35 2005 GMT
    Serial Number: FB54DE119E8E3526
        Revocation Date: Dec 25 17:19:24 2005 GMT
    Signature Algorithm: md5WithRSAEncryption
    48:16:eb:0c:0d:0a:e4:0a:a3:dc:25:f3:98:2b:ef:06:65:6b:
    db:ff:23:ed:93:f2:74:7f:c2:52:e3:1a:cb:fe:14:93:3f:a5:
    8b:5c:e9:d2:5c:c3:0f:99:3f:f1:44:fe:fe:61:85:b0:ed:c2:
    83:51:26:d7:26:c7:89:c4:05:d3:76:b8:08:a0:b8:9a:cf:d0:
    02:a0:23:2b:ac:1c:1a:48:11:c3:d1:9e:28:75:05:cc:c3:55:
    9d:16:bb:a1:a6:be:25:68:cf:45:71:e8:3c:5d:3b:19:f8:5b:
    0f:d5:d0:ee:b5:26:4b:63:3a:3e:50:2d:74:b4:67:03:5d:0a:
    e4:92

```

រូបភាព 4.4 Certificate Revocation List (CRL)